

I. Code submission and result reproducibility

- Please refer to the README.md in the repository codebase.
- Trained model: model.tar

II. Basic information

- Kotaro Watanabe
- Tokyo, Japan
- Please use the attached icon.png.

III. Model documentation and write-up

1. Application engineer at a meeting transcription SaaS company.
2. Skill development.
3. Data consists of two components — noise and useful signal — but it was difficult to draw a clear boundary between them. I prepared multiple datasets by varying the data filtering threshold, using WER from a fine-tuned Parakeet model as the criterion. I then performed checkpoint ensembling of Qwen-3-ASR-1.7b fine-tuned with LoRA on each of those datasets.
4. Please refer to the scores in the repository.
5. Three most impactful parts of the code.

(1) WER/CER-based data filtering — [csrc/src/csrc/filters.py](#)

Filters the training set by age-bucket-specific WER/CER thresholds based on Parakeet predictions. Varying the thresholds produced the different training subsets used for the ensemble.

```
Python
import pandas as pd

def apply_filter(
    df: pd.DataFrame,
    cer_thresholds: dict[str, float | None],
```

```

wer_thresholds: dict[str, float | None] | None = None,
filter_mode: str = "and",
) -> pd.DataFrame:
    """Apply age-bucket-specific CER/WER thresholds and return the filtered
    DataFrame."""
    original_count = len(df)
    mask = pd.Series(False, index=df.index)

    all_buckets = set(cer_thresholds.keys())
    if wer_thresholds is not None:
        all_buckets |= set(wer_thresholds.keys())

    for bucket in sorted(all_buckets):
        cer_threshold = cer_thresholds.get(bucket)
        wer_threshold = wer_thresholds.get(bucket) if wer_thresholds is not None
    else None
        bucket_mask = df["age_bucket"] == bucket

        passed = bucket_mask
        cer_cond = df["cer"] <= cer_threshold if cer_threshold is not None else
None
        wer_cond = df["wer"] <= wer_threshold if wer_threshold is not None else
None

        if cer_cond is not None and wer_cond is not None:
            combined = (cer_cond | wer_cond) if filter_mode == "or" else (cer_cond
& wer_cond)
            passed = passed & combined
        elif cer_cond is not None:
            passed = passed & cer_cond
        elif wer_cond is not None:
            passed = passed & wer_cond

        mask = mask | passed

    unknown_mask = ~df["age_bucket"].isin(all_buckets)
    if unknown_mask.any():
        mask = mask | unknown_mask

    df = df[mask].reset_index(drop=True)
    filtered_count = original_count - len(df)
    print(f"Filter(total): {original_count:,} -> {len(df):,}
(-{filtered_count:,})")
    return df

```

(2) LoRA adapter averaging — `csrc_qwen/src/csrc_qwen/average_adapters.py`

Averages N LoRA adapters element-wise. Used both for checkpoint averaging and for averaging adapters across runs with different filter thresholds.

Python

```
import json
from pathlib import Path
import torch
from safetensors.torch import load_file as safe_load_file
from safetensors.torch import save_file as safe_save_file

def _validate_adapter_configs(paths: list[Path]) -> None:
    configs = []
    for p in paths:
        with (p / "adapter_config.json").open() as f:
            configs.append(json.load(f))
    base = configs[0]
    for i, cfg in enumerate(configs[1:], start=2):
        for key in ("r", "lora_alpha", "target_modules", "peft_type"):
            v1, v2 = base.get(key), cfg.get(key)
            if isinstance(v1, list) and isinstance(v2, list):
                v1, v2 = sorted(v1), sorted(v2)
            if v1 != v2:
                raise ValueError(f"adapter_config.json '{key}' mismatch: {v1} vs {v2}")

def _average_state_dicts(state_dicts: list[dict[str, torch.Tensor]]) -> dict[str, torch.Tensor]:
    base_keys = state_dicts[0].keys()
    averaged: dict[str, torch.Tensor] = {}
    for key in base_keys:
        stacked = torch.stack([sd[key] for sd in state_dicts])
        averaged[key] = stacked.mean(dim=0)
    return averaged

def main(adapter_paths: list[str], output_path: str) -> None:
    dirs = [Path(p) for p in adapter_paths]
    _validate_adapter_configs(dirs)
    state_dicts = [
        safe_load_file(str(d / "adapter_model.safetensors"), device="cpu") for d
    in dirs
    ]
    averaged = _average_state_dicts(state_dicts)
    out = Path(output_path)
    out.mkdir(parents=True, exist_ok=True)
    safe_save_file(averaged, str(out / "adapter_model.safetensors"))
```

(3) Full-model averaging — `csrc_qwen/src/csrc_qwen/average_models.py`

Averages full Qwen3-ASR models after each LoRA-averaged adapter has been merged. Combined with (2), gives a two-stage ensemble at single-model inference cost.

```
Python
import json
from pathlib import Path
import torch
from safetensors.torch import load_file as safe_load_file
from safetensors.torch import save_file as safe_save_file

def _load_state_dict(model_dir: Path) -> dict[str, torch.Tensor]:
    index_path = model_dir / "model.safetensors.index.json"
    single_path = model_dir / "model.safetensors"
    if single_path.exists():
        return safe_load_file(str(single_path), device="cpu")
    if index_path.exists():
        with index_path.open() as f:
            index = json.load(f)
            shard_files = set(index["weight_map"].values())
            state_dict: dict[str, torch.Tensor] = {}
            for shard in sorted(shard_files):
                state_dict.update(safe_load_file(str(model_dir / shard),
device="cpu"))
            return state_dict
    raise FileNotFoundError(model_dir)

def main(model_paths: list[str], output_path: str, weights: list[float] | None =
None) -> None:
    dirs = [Path(p) for p in model_paths]
    n = len(dirs)
    if weights is None:
        weights = [1.0 / n] * n

    state_dicts = [_load_state_dict(d) for d in dirs]
    ref_keys = state_dicts[0].keys()
    averaged: dict[str, torch.Tensor] = {}
    for key in ref_keys:
        averaged[key] = sum(w * sd[key] for w, sd in zip(weights, state_dicts))

    out = Path(output_path)
    out.mkdir(parents=True, exist_ok=True)
    safe_save_file(averaged, str(out / "model.safetensors"))
```

See [form/solution.md](#) for the broader methodology and ablation results.

6. Machine Specs
 - CPU (model): Ryzen 7 5800X
 - GPU (model or N/A): RTX 5090
 - Memory (GB): 32GB
 - OS: Ubuntu 24.04
 - Train duration: parakeet:10-20h, qwen:20-35h
 - Inference duration: 25-45 minutes in the competition environment.
7. Local smoke run scores were generally consistent with submission smoke run results.
8. None.
9. None. WER/CER only.
10. I applied additive noise augmentation using real-class audio, but it did not yield any meaningful improvement.
11. Training exclusively on clean audio with aggressive filtering makes the model less robust to noisy inputs, while training without any filtering failed to improve smoke scores. A more appropriate data cleaning strategy likely exists and is worth exploring.
12. The solution is already fairly lightweight, consisting of a single Qwen-3-ASR-1.7b with vLLM. When audio sorting was applied, inference completed in approximately 25 minutes in the competition environment; however, an OOM error occurred partway through the competition. Disabling the sorting brought inference time back up to around 45 minutes. Since the inference code itself was not changed, the root cause of the OOM is unclear, but it is likely sensitive to vLLM configuration parameters.